architecture design architecture design

● Let's complete the architecture design by defining the data flow between components.

● Update Todos □ □ Design frontend components □ Design backend server architecture □ Plan content processing pipeline □ Design PDF generation module □ Define data flow between components

● Data Flow Between Components

1. User Request Flow

   User → Frontend → API Gateway → Job Orchestrator → Queue System

- User Input Data:
  - Target URL
  - Crawl depth
  - Content filter preferences
  - Template selection
  - Custom styling options
- Job Metadata:
  - JobID (unique identifier)
  - UserID (for authenticated users)
  - Timestamp
  - Status (pending, processing, completed, failed)

2. Content Processing Flow

   Queue System → Crawler → Content Processor → Temporary Storage

- Crawler Data:
  - URL graph (pages to crawl)
  - Robots.txt rules
  - Crawl progress (URLs visited/pending)
- Page Content Data:
  - HTML content (raw)
  - Extracted main content (cleaned)
  - Metadata (title, description, publication date)
  - Media references (images, videos)
  - Hierarchical structure (headings, sections)
- Storage Data:
  - Structured content JSON
  - Downloaded media files
  - Processing logs
  - Page relationships

3. Progress Update Flow

   Processing Pipeline → WebSocket Server → Frontend

- Progress Events:
  - Job status changes
  - Pages crawled (count, percentage)
  - Content processed (count, percentage)

    - Errors encountered
    - Estimated time remaining

4. PDF Generation Flow

   Queue System → PDF Generator → Storage Service → Frontend

- PDF Configuration:
  - Template data
  - Custom styling parameters
  - Output preferences
- Generated Assets:
  - Preview PDF (low resolution)
  - Final PDF
  - Thumbnail images
  - Metadata for library display

5. User Management Flow

   Frontend → Authentication Service → Database

- User Data:
  - Authentication credentials
  - Profile information
  - Preferences
  - Usage statistics
- Library Data:
  - Saved PDFs
  - Job history
  - Templates

6. Key Data Models

7. Job Model interface Job { id: string; userId: string | null; // null for anonymous status: 'pending' | 'processing' | 'completed' | 'failed'; sourceUrl: string; depth: number; configuration: { contentFilters: ContentFilterOptions; outputFormat: OutputFormat; template: TemplateConfig; customStyles: StyleOptions; }; progress: { crawlProgress: number; // 0-100 processingProgress: number; // 0-100 pdfGenerationProgress: number; // 0-100 }; results: { previewUrl?: string; downloadUrl?: string; pageCount?: number; fileSize?: number; }; errors: ErrorRecord[]; createdAt: Date; updatedAt: Date; completedAt?: Date; }

8. Page Model interface Page { id: string; jobId: string; url: string; depth: number; title: string; content: ProcessedContent; metadata: { originalHtml: string; extractionMethod: string; processedAt: Date; }; assets: Asset[]; links: Link[]; errors: ErrorRecord[]; }

9. ProcessedContent Model interface ProcessedContent { elements: ContentElement[]; structure: { headings: HeadingNode[]; sections: SectionNode[]; }; textContent: string; // Plain text for searching }

10. PDF Model interface PDF { id: string; jobId: string; userId: string | null; title: string; description?: string; templateId: string; metadata: { sourceUrl:

string; createdAt: Date; pageCount: number; wordCount: number; }; storage: { previewPath: string; fullPath: string; thumbnailPath: string; size: number; }; isPublic: boolean; downloadCount: number; }

11. User Model interface User { id: string; email: string; name?: string; authProvider: 'email' | 'google' | 'github'; preferences: UserPreferences; usage: { totalJobs: number; totalPdfs: number; storageUsed: number; }; createdAt: Date; lastLoginAt: Date; }

12. Data Flow Optimizations

13. Streaming Processing:

    - Process content as it's crawled rather than waiting for all crawling to complete
    - Enable partial preview generation

14. Caching Strategy:

    - Cache processed content for repeated generation with different templates
    - Cache common assets (images, styles) across jobs

15. Progressive Loading:

    - Send processed content to frontend incrementally
    - Update preview as more content becomes available

16. Persistent Connections:

    - Use WebSockets for real-time progress updates
    - Maintain connection status and reconnect as needed

17. Background Processing:

    - Process large jobs in background even if user disconnects
    - Send email notification upon completion ● Let’s complete the architecture design by defining the data flow between components.

● Update Todos □ □ Design frontend components □ Design backend server architecture □ Plan content processing pipeline □ Design PDF generation module □ Define data flow between components

● Data Flow Between Components

User Request Flow User → Frontend → API Gateway → Job Orchestrator → Queue System

User Input Data: Target URL Crawl depth Content filter preferences Template selection Custom styling options Job Metadata: JobID (unique identifier) UserID (for authenticated users) Timestamp Status (pending, processing, completed, failed) Content Processing Flow Queue System → Crawler → Content Processor → Temporary Storage

Crawler Data: URL graph (pages to crawl) Robots.txt rules Crawl progress (URLs visited/pending) Page Content Data: HTML content (raw) Extracted main content (cleaned) Metadata (title, description, publication date) Media references (images, videos) Hierarchical structure (headings, sections) Storage Data: Structured content JSON Downloaded media files Processing logs Page relationships Progress Update Flow Processing Pipeline → WebSocket Server → Frontend

Progress Events: Job status changes Pages crawled (count, percentage) Content processed (count, percentage) Errors encountered Estimated time remaining PDF Generation Flow Queue System → PDF Generator → Storage Service → Frontend

PDF Configuration: Template data Custom styling parameters Output preferences Generated Assets: Preview PDF (low resolution) Final PDF Thumbnail images Metadata for library display User Management Flow Frontend → Authentication Service → Database

User Data: Authentication credentials Profile information Preferences Usage statistics Library Data: Saved PDFs Job history Templates Key Data Models

Job Model interface Job { id: string; userId: string | null; // null for anonymous status: ‘pending’ | ‘processing’ | ‘completed’ | ‘failed’; sourceUrl: string; depth: number; configuration: { contentFilters: ContentFilterOptions; outputFormat: OutputFormat; template: TemplateConfig; customStyles: StyleOptions; }; progress: { crawlProgress: number; // 0-100 processingProgress: number; // 0-100 pdfGenerationProgress: number; // 0-100 }; results: { previewUrl?: string; downloadUrl?: string; pageCount?: number; fileSize?: number; }; errors: ErrorRecord[]; createdAt: Date; updatedAt: Date; completedAt?: Date; }

Page Model interface Page { id: string; jobId: string; url: string; depth: number; title: string; content: ProcessedContent; metadata: { originalHtml: string; extractionMethod: string; processedAt: Date; }; assets: Asset[]; links: Link[]; errors: ErrorRecord[]; }

ProcessedContent Model interface ProcessedContent { elements: ContentElement[]; structure: { headings: HeadingNode[]; sections: SectionNode[]; }; textContent: string; // Plain text for searching }

PDF Model interface PDF { id: string; jobId: string; userId: string | null; title: string; description?: string; templateId: string; metadata: { sourceUrl: string; createdAt: Date; pageCount: number; wordCount: number; }; storage: { previewPath: string; fullPath: string; thumbnailPath: string; size: number; }; isPublic: boolean; downloadCount: number; }

User Model interface User { id: string; email: string; name?: string; authProvider: ‘email’ | ‘google’ | ‘github’; preferences: UserPreferences; usage: { totalJobs: number; totalPdfs: number; storageUsed: number; }; createdAt: Date; lastLoginAt: Date; }

Data Flow Optimizations

Streaming Processing:

- Process content as it's crawled rather than waiting for all crawling to complete
- Enable partial preview generation Caching Strategy:
- Cache processed content for repeated generation with different templates
- Cache common assets (images, styles) across jobs Progressive Loading:
- Send processed content to frontend incrementally
- Update preview as more content becomes available Persistent Connections:
- Use WebSockets for real-time progress updates
- Maintain connection status and reconnect as needed Background Processing:
- Process large jobs in background even if user disconnects
- Send email notification upon completion Markdown selection 5024 bytes 615 words 204 lines Ln 205, Col 45HTML 3621 characters 548 words 178 paragraphs